SPRING BOOT

Interview Preparation Guide

Level IV (Advanced) & Level V (Expert)

106+	Detailed	Cheat
Questions	Answers	Sheet Included

⭐ Yellow highlighted questions = Frequently Asked in Interviews

Covers: Auto-configuration | Transactions | Testing | Reactive | Caching | Security | Docker | Kafka | Tracing | Best Practices

Spring Boot 3.x | Java 17/21 | Enterprise Grade

Table of Contents

Step 12 – Spring Boot Level IV (Advanced)

This section covers 76 advanced Spring Boot questions. Questions highlighted in yellow (⭐) are frequently asked in technical interviews.

Q1 ⭐ [FREQUENTLY ASKED]: How does the Spring Boot auto-configuration mechanism work, and how can it be overridden?

Spring Boot auto-configuration automatically configures application beans based on the classpath and existing beans.

- Spring Boot reads META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports to discover auto-configuration classes
- Each auto-config class uses @Conditional annotations to decide whether to apply configuration
- Common conditionals: @ConditionalOnClass, @ConditionalOnMissingBean, @ConditionalOnProperty

How to Override:

- Define your own @Bean of the same type — @ConditionalOnMissingBean skips auto-config
- Use spring.autoconfigure.exclude property to disable specific auto-configuration
- Use @EnableAutoConfiguration(exclude={DataSourceAutoConfiguration.class})

```
spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
```

Q2 ⭐ [FREQUENTLY ASKED]: If you were tasked with ensuring high availability for a Spring Boot e-commerce application during peak times, what architectural decisions would you make?

Key architectural decisions for high availability:

- Horizontal Scaling: Deploy multiple instances behind a load balancer (AWS ALB / NGINX)
- Database: Use read replicas for

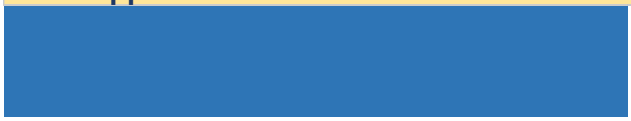
SELECT queries; connection pooling via HikariCP

- Caching: Redis/Hazelcast for session data, product catalog, pricing
- Circuit Breaker: Resilience4j to handle downstream failures gracefully
- Async Processing: RabbitMQ/Kafka for order processing, email notifications
- CDN: CloudFront for static assets to reduce server load
- Auto-scaling: Kubernetes HPA or AWS Auto Scaling Groups
- Health checks: Spring Boot Actuator /actuator/health for load balancer probes
- Zero-downtime deploys: Blue-green or rolling deployments

Q3: Your Spring Boot application suddenly needs to support twice the user load. What immediate steps would you take?

- Immediate: Add more instances behind the load balancer (horizontal scale-out)
- Enable caching with @EnableCaching + Redis to reduce DB hits
- Check and tune HikariCP connection pool size (spring.datasource.hikari.maximum-pool-size)
- Profile the app with APM tools (New Relic, Datadog) to identify bottlenecks
- Add database indexes on frequently queried columns
- Enable HTTP response compression (server.compression.enabled=true)
- Use async processing (@Async) for non-critical background tasks
- Review and optimize N+1 query problems with JOIN FETCH or batch fetching

Q4 ⭐ [FREQUENTLY ASKED]: What testing strategies do you recommend for Spring Boot applications?



A layered testing strategy (Testing Pyramid):

- Unit Tests: JUnit 5 + Mockito — test

individual methods/classes in isolation

- Slice Tests: @WebMvcTest for controllers, @DataJpaTest for repositories, @JsonTest for JSON
- Integration Tests: @SpringBootTest — load full context, test component interaction
- Contract Tests: Spring Cloud Contract — verify API contracts between services
- End-to-End Tests: Selenium/Testcontainers for browser or real DB testing

```
@WebMvcTest(UserController.class)
class UserControllerTest { @Autowired
MockMvc mockMvc; }
```

- Use Testcontainers to spin up real databases in tests for full integration coverage

Q5 ⭐ [FREQUENTLY ASKED]: Can you describe a scenario where you can implement asynchronous messaging in a Spring Boot application?

Scenario: Order Processing in an E-Commerce Application

- When a user places an order, the HTTP response is returned immediately after saving the order
- Order fulfillment, email confirmation, inventory update, and payment processing happen asynchronously via message queues
- RabbitMQ or Kafka acts as the message broker

```
@RabbitListener(queues =
"order.queue")
public void processOrder(Order order)
{ /* fulfillment logic */ }
```

- Benefits: Decoupling, resilience (messages survive restarts), throughput improvement
- Other scenarios: Email/SMS notifications, report generation, data synchronization

Q6 ⭐ [FREQUENTLY ASKED]: You need to migrate an existing application to use a new database schema in Spring Boot without downtime. How would you plan and execute this migration?

Strategy: Expand-Contract (Blue-Green

Schema Migration)

- Step 1 — Expand: Add new columns/tables without removing old ones; deploy code that writes to both old and new
- Step 2 — Migrate: Run background data migration scripts to populate new columns
- Step 3 — Contract: Once verified, remove old columns/code in a subsequent release
- Use Flyway or Liquibase for versioned, repeatable migrations

```
spring.flyway.enabled=true
spring.flyway.locations=classpath:db/migration
```

- Use feature flags to toggle between old and new data access paths
- Always test migrations on a copy of production data first

Q7 ⭐ [FREQUENTLY ASKED]: How do you achieve logging in Spring Boot?

Spring Boot uses SLF4J as the logging facade with Logback as the default implementation.

- No additional dependency needed — spring-boot-starter includes spring-boot-starter-logging
- Configure via application.properties:

```
logging.level.root=WARN
logging.level.com.myapp=DEBUG
logging.file.name=app.log
```

- Use Logger in your class:

```
private static final Logger log = LoggerFactory.getLogger(MyClass.class);
```

- Or use Lombok's @Slf4j annotation to auto-generate the logger field
- For custom patterns, create logback-spring.xml in resources

Q8: What is SLF4J logging?

SLF4J (Simple Logging Facade for Java) is an abstraction layer for various logging frameworks.

- It is NOT a logging implementation — it is a facade/API

- Allows you to plug in different implementations: Logback, Log4j2, java.util.logging
- Your code depends only on SLF4J API; the actual logger is swapped at runtime
- Supports parameterized logging to avoid string concatenation overhead:

```
log.debug("User {} logged in at {}", userId, timestamp);
```

- Spring Boot defaults to Logback as the SLF4J binding

Q9: If you have to switch between Logback to Log4j2, what changes are required in the code?

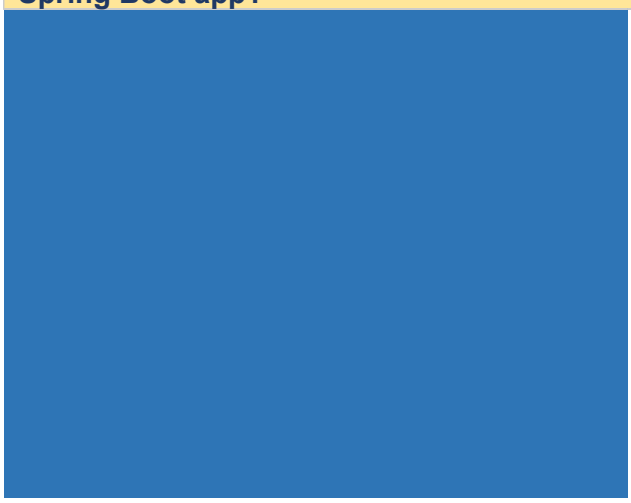
- In pom.xml, exclude Logback from spring-boot-starter-logging and add Log4j2 starter:

```
<exclusions><exclusion><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-logging</artifactId></exclusion></exclusions>

<dependency><groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-log4j2</artifactId></dependency>
```

- Add log4j2-spring.xml or log4j2.xml in src/main/resources
- No Java code changes needed — SLF4J API stays the same
- Remove any logback-spring.xml or logback.xml files

Q10 ⭐ [FREQUENTLY ASKED]: How do you achieve multiple DB connections in a Spring Boot app?



- Define multiple DataSource beans with @Primary on the default one
- Use @ConfigurationProperties to bind each datasource from properties

```
spring.datasource.primary.url=jdbc:mysql://...
spring.datasource.secondary.url=jdbc:postgresql://...
```

- Create separate JPA configuration classes with @EnableJpaRepositories(basePackages=..., entityManagerFactoryRef=...)
- Use @Qualifier to inject the correct DataSource where needed

- Route requests dynamically using AbstractRoutingDataSource for read/write splitting

Q11 ⭐ [FREQUENTLY ASKED]: How does Spring Boot handle database migrations?

Spring Boot provides first-class integration with Flyway and Liquibase.

- Flyway: Add spring-boot-starter-flyway; place SQL scripts in db/migration/ with naming V1__desc.sql
- Liquibase: Add spring-boot-starter-liquibase; use XML/YAML changelog files
- Migrations run automatically at application startup before the app is ready
- Flyway tracks applied migrations in a flyway_schema_history table

```
spring.flyway.enabled=true
spring.flyway.baseline-on-migrate=true
```

- Supports repeatable migrations (R__), undo migrations, and callbacks

Q12 ⭐ [FREQUENTLY ASKED]: What mechanisms does Spring Boot provide for transaction management?

- @Transactional annotation — declarative transaction management on class or method
- PlatformTransactionManager — programmatic transaction management
- TransactionTemplate — for fine-grained programmatic control
- Propagation levels: REQUIRED (default), REQUIRES_NEW, NESTED, SUPPORTS, etc.
- Isolation levels: READ_COMMITTED, REPEATABLE_READ, SERIALIZABLE

```
@Transactional(propagation =
Propagation.REQUIRES_NEW, isolation =
Isolation.SERIALIZABLE)
```

- rollbackFor and noRollbackFor attributes control which exceptions trigger rollback
- Spring Boot auto-configures DataSourceTransactionManager for JDBC or JpaTransactionManager for JPA

Q13: Can transaction management be externally managed in Spring Boot, or must it be within the application?

- Yes — transactions can be externally managed via JTA (Java Transaction API)
- Spring Boot supports Atomikos and Bitronix as JTA transaction managers
- Add spring-boot-starter-jta-atomikos for distributed transactions across multiple resources
- Application server (JBoss, WebLogic) can also provide the JTA implementation externally

```
spring.jta.enabled=true
```

- Useful for XA transactions spanning multiple databases or a database + JMS broker

Q14 ⭐ [FREQUENTLY ASKED]: You are designing an e-commerce application requiring precise control over transactions. What approach would you take in Spring Boot?

- Use @Transactional with explicit propagation and isolation per business operation
- Order placement: REQUIRES_NEW isolation=SERIALIZABLE to prevent phantom reads on inventory
- Payment processing: REQUIRES_NEW so payment always rolls back independently
- Report generation: SUPPORTS or NOT_SUPPORTED to avoid holding transactions
- Use optimistic locking with @Version on entities to handle concurrent updates

```
@Transactional(propagation =
Propagation.REQUIRES_NEW, rollbackFor
= PaymentException.class)
```

- Use saga pattern for distributed transactions across microservices
- Implement idempotency keys for payment to prevent double charges on retry

Q15 ⭐ [FREQUENTLY ASKED]: How do you handle form validation in Spring Boot applications?

- Add spring-boot-starter-validation

dependency (includes Hibernate Validator)

- Annotate DTO/model fields with Bean Validation annotations:

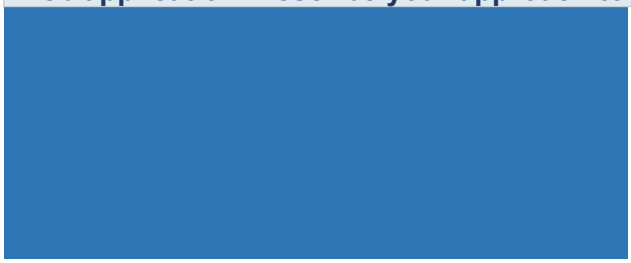
```
@NotBlank @Email private String email;
@Min(0) @Max(150) private int age;
```

- In controller, use @Valid or @Validated on the @RequestBody or @ModelAttribute parameter
- Handle MethodArgumentNotValidException with @ExceptionHandler or @ControllerAdvice
- Validation groups allow different rules for create vs update operations
- Custom validation: implement ConstraintValidator<YourAnnotation, FieldType>

Q16: Can you integrate third-party libraries for form validation? How?

- Yes — any library implementing JSR-380 (Bean Validation API) can be integrated
- Add the library JAR as a Maven/Gradle dependency
- Spring Boot auto-detects the Validator bean on the classpath
- For non-standard libraries, create a @Bean of type Validator and Spring will use it
- Example: OWASP Anti-Samy for HTML input sanitization alongside Bean Validation
- Custom ConstraintValidatorFactory can inject Spring beans into custom validators

Q17: You're tasked with validating user input across multiple forms in a Spring Boot web application. Describe your approach to maintain consistency.



- Create shared DTOs/request objects annotated with Bean Validation constraints
- Define validation groups (interface markers) for Create, Update, Delete scenarios
- Use a centralized @ControllerAdvice

with @ExceptionHandler for validation errors

- Return consistent error response structure (field, message, code)
- Cross-field validation using class-level @Constraint annotations
- Document validation rules in OpenAPI/Swagger annotations for API consumers
- Write unit tests for each validation constraint to prevent regression

Q18 ⭐ [FREQUENTLY ASKED]: What are the ways to deploy a Spring Boot application?

- Fat JAR (Executable JAR): java -jar app.jar — most common; embeds server
- WAR file: Deploy to external Tomcat, JBoss, WildFly
- Docker Container: Containerize the JAR; run on Kubernetes, ECS, etc.
- Cloud Platforms: AWS Elastic Beanstalk, Azure App Service, Google Cloud Run
- System Service: Register as a Linux systemd service for server deployments
- Kubernetes: Deploy using Helm charts or plain YAML manifests with health probes
- Serverless: AWS Lambda via Spring Cloud Function

Q19 ⭐ [FREQUENTLY ASKED]: How does Spring Boot simplify the deployment process compared to traditional Spring applications?

- Embedded Server: Tomcat/Jetty/Undertow baked into the JAR — no server installation needed
- Fat JAR: Single self-contained artifact; no complex WAR + server setup
- Auto-configuration: No need to manually configure DataSource, EntityManager, etc.
- Externalized configuration: Profiles, environment variables, config servers — no rebuild
- Actuator: Built-in health, metrics, info endpoints for ops teams

- Spring Boot Maven/Gradle plugin: mvn spring-boot:build-image to create OCI images

Q20 ⭐ [FREQUENTLY ASKED]: Discuss the support for reactive programming in Spring Boot.

Spring Boot supports reactive programming via Spring WebFlux (added in Spring 5).

- WebFlux uses Project Reactor — Mono<T> (0-1 item) and Flux<T> (0-N items)
- Non-blocking I/O on Netty or Servlet 3.1+ containers
- Reactive data access: Spring Data R2DBC (relational), Reactive MongoDB, Redis
- @RestController works with WebFlux — return Mono/Flux instead of plain objects

```
@GetMapping("/users")
public Flux<User> getUsers() { return
userRepo.findAll(); }
```

- Suitable for high-concurrency with lower thread count vs traditional blocking
- Not recommended for CPU-intensive workloads or simple CRUD

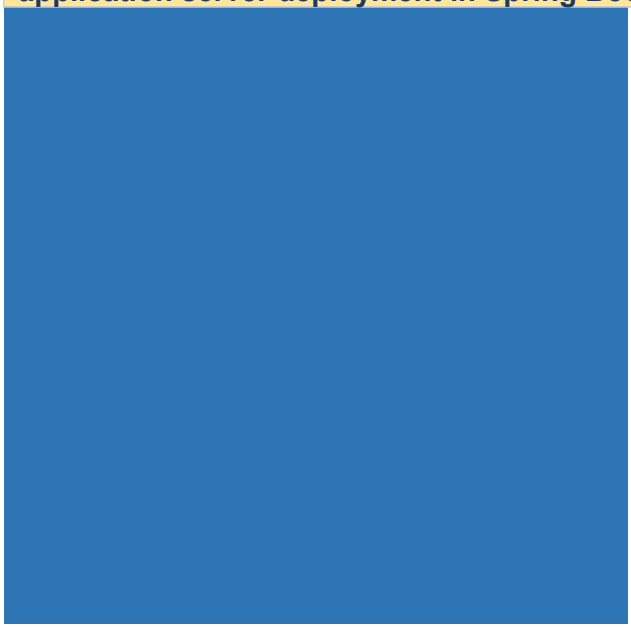
Q21: How does Spring Boot handle back pressure in reactive streams?

- Back pressure is built into Project Reactor via the Reactive Streams specification
- Subscribers request a specific number of items (request(n)) from the publisher
- Operators like limitRate(), buffer(), onBackpressureBuffer(), onBackpressureDrop() manage flow

```
Flux.range(1, 1000).limitRate(10) //
downstream only gets 10 at a time
```

- WebFlux propagates back pressure from client (via HTTP/2 flow control or SSE)
- Prevents OutOfMemoryError by ensuring producers don't overwhelm consumers

Q22 ⭐ [FREQUENTLY ASKED]: What is the difference between embedded and external application server deployment in Spring Boot?



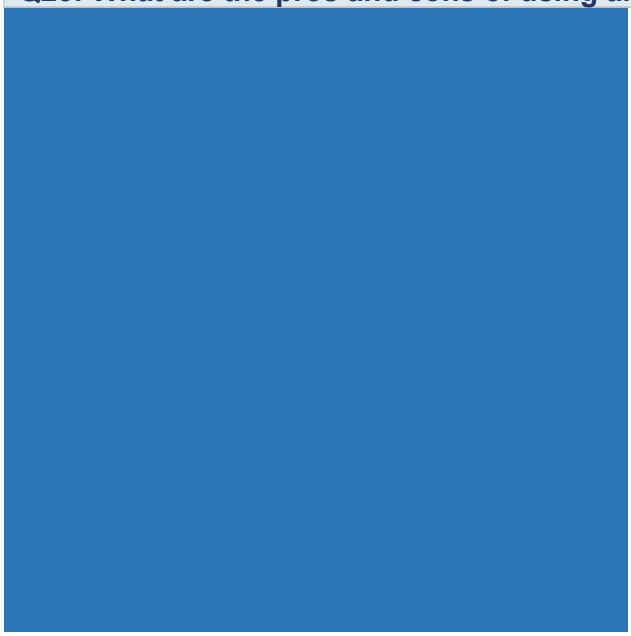
Embedded Server:

- Server (Tomcat/Jetty) is packaged inside the JAR
- Run with: java -jar app.jar — self-contained, no external install
- Default and recommended for microservices and cloud deployments

External Server:

- Package as WAR; deploy to standalone Tomcat, JBoss, WebLogic
- Extend SpringBootServletInitializer and override configure()
- Requires changing packaging to war in pom.xml
- Useful for enterprise environments with centralized server management

Q23: What are the pros and cons of using an embedded server in Spring Boot?



Pros:

- Self-contained deployment — no server installation required
- Consistent environments across dev/test/prod
- Simple CI/CD pipeline — single JAR artifact
- Easier to run locally and in containers

Cons:

- Larger JAR size (Tomcat adds ~2MB)
- No shared server management across multiple applications
- Less fine-grained server configuration compared to standalone servers
- Enterprise certifications may require specific standalone server versions

Q24: You need to migrate an application from an embedded Tomcat to an external Tomcat server. What steps would you follow?

- Change packaging from jar to war in pom.xml
- Extend SpringBootServletInitializer in your main class and override configure():

```
public class Application extends
```

```
SpringBootServletInitializer {
  @Override
  protected SpringApplicationBuilder
configure(SpringApplicationBuilder b)
{
    return
b.sources(Application.class);
  }
}
```

- Mark embedded Tomcat dependency as provided:

```
<dependency><artifactId>spring-boot-
starter-
tomcat</artifactId><scope>provided</sc
ope></dependency>
```

- Build WAR: mvn package → target/app.war
- Deploy to external Tomcat's webapps/ directory
- Update context paths and environment variables accordingly

Q25: You have to deploy a Spring Boot application on both AWS and Azure. What would be your approach for each?

AWS Deployment:

- Build Docker image → push to ECR → deploy to ECS/EKS or Elastic Beanstalk
- Use AWS Parameter Store or Secrets Manager for config; Spring Cloud AWS for integration
- RDS for database, ElastiCache (Redis) for caching, SQS/SNS for messaging

Azure Deployment:

- Push Docker image to Azure Container Registry → deploy to Azure Container Apps or AKS
- Azure App Configuration + Key Vault for externalized config
- Azure Database for PostgreSQL/MySQL, Azure Cache for Redis

Common:

- Spring profiles: aws, azure — activate per environment via env variable
- Terraform or Bicep for infrastructure as code

Q26: You need to build a highly scalable real-time data processing application. How

would you leverage Spring Boot's reactive features?

- Use Spring WebFlux with Netty for non-blocking HTTP endpoints
- Integrate Spring Kafka or Spring Cloud Stream for real-time event ingestion
- Use R2DBC for non-blocking reactive database access
- Use Flux.fromIterable() or Flux.create() for streaming data pipelines
- Apply back pressure with limitRate() and buffer operators
- Use Project Reactor schedulers: Schedulers.parallel() for CPU, Schedulers.boundedElastic() for I/O
- SSE (Server-Sent Events) for streaming data to browser clients

```
@GetMapping(produces =
MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<Event> stream() { return
eventService.getEventStream(); }
```

Q27 ⭐ [FREQUENTLY ASKED]: Your application has high read operations and needs efficient caching strategies. What caching solutions would you consider with Spring Boot?

- Spring Cache Abstraction: @EnableCaching + @Cacheable, @CachePut, @CacheEvict annotations
- Redis (Recommended): Distributed, supports TTL, persistence, pub/sub; spring-boot-starter-data-redis
- Caffeine: In-memory, high-performance local cache for single-instance apps
- Hazelcast: Distributed in-memory grid — good for session clustering and large datasets
- EhCache: Mature, supports disk overflow for large caches

```
@Cacheable(value = "products", key =
"#id", unless = "#result == null")
public Product getProduct(Long id)
{ ... }
```

- Multi-level caching: L1 Caffeine (local) + L2 Redis (distributed) for best performance

Q28: What are the disadvantages of Spring Boot's default caching mechanism?

- Default is ConcurrentHashMap-based

— in-memory only, not distributed

- No TTL (Time-To-Live) support out of the box with simple cache
- Cache is lost on application restart
- Not suitable for multi-instance deployments — each instance has its own cache
- No cache statistics or monitoring in simple cache
- No eviction policies (LRU, LFU) in the default implementation
- Solution: Replace with Redis or Caffeine using spring.cache.type=redis/caffeine

Q29: How does Spring Boot simplify the process of creating Docker images?

- Spring Boot Maven/Gradle plugin supports buildpacks (Cloud Native Buildpacks):

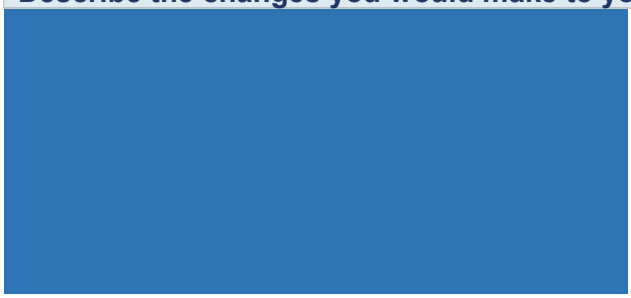
```
mvn spring-boot:build-image -Dspring-boot.build-image.imageName=myapp:latest
```

- No Dockerfile needed — buildpacks detect the runtime and create an optimized OCI image
- Layered JARs: spring-boot:repackage creates layered JARs for better Docker layer caching
- Dockerfile approach: COPY the layered JAR, EXPOSE port, ENTRYPOINT java -jar

```
FROM eclipse-temurin:21-jre
COPY target/app.jar app.jar
ENTRYPOINT ["java","-jar","/app.jar"]
```

- Jib plugin (Google): Build Docker images without a Docker daemon

Q30: You are moving your Spring Boot application to a Docker-based environment. Describe the changes you would make to your deployment process.



- Create a multi-stage Dockerfile: build stage (Maven/Gradle) + runtime stage (JRE)
- Externalize ALL configuration via environment variables or config maps (12-factor app)
- Add Docker health check using Actuator: HEALTHCHECK CMD curl

/actuator/health

- Use Docker Compose for local development with dependent services (DB, Redis)
- Set up container registry (ECR, Docker Hub, ACR) for image storage
- Define resource limits (CPU, memory) in Docker run or Kubernetes manifests
- Use .dockerignore to exclude target/, .git, etc. from build context
- Configure logging to stdout (JSON format) for container log aggregation

Q31: You are designing a system where it is critical to have only one instance of a configuration manager. How would you implement the Singleton pattern?

- In Spring, all @Bean definitions are Singleton scope by default — no extra pattern needed
- Simply define the ConfigurationManager as a @Component or @Bean and inject it everywhere
- Spring IoC container ensures single instance per application context

```
@Component
public class ConfigurationManager {
  // Only one instance created by
Spring
}
```

- If needed outside Spring, use thread-safe Java Singleton:

```
private static volatile
ConfigurationManager INSTANCE;
public static ConfigurationManager
getInstance() {
  if (INSTANCE == null)
{ synchronized(ConfigurationManager.cl
ass) {
    if (INSTANCE == null) INSTANCE =
new ConfigurationManager(); } }
  return INSTANCE;
}
```

Q32 ⭐ [FREQUENTLY ASKED]: What is the purpose of the @RequestBody annotation?

- @RequestBody deserializes the HTTP request body into a Java object
- Spring uses HttpMessageConverter (Jackson by default) to parse JSON/XML

```
@PostMapping("/users")
```

```
public User create(@RequestBody @Valid
UserRequest request) { ... }
```

- Works with Content-Type: application/json — reads the entire body stream
- Without @RequestBody, a parameter is read from query params or path variables
- Compare: @RequestParam reads from URL query string; @PathVariable from URL path
- Can be combined with @Valid to trigger Bean Validation on the deserialized object

Q33 ⭐ [FREQUENTLY ASKED]: Describe the process of creating a custom Spring Boot starter. Why might this be useful?

A custom starter bundles auto-configuration + dependencies for easy reuse.

- Create two modules: starter (pom only with dependencies) and autoconfigure (logic)
- In autoconfigure, write @Configuration class with @ConditionalOnClass, @ConditionalOnProperty
- Register in META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
- Include default properties via @ConfigurationProperties class

```
// autoconfigure module
@AutoConfiguration
@ConditionalOnClass(SomeLibrary.class)
public class
MyLibraryAutoConfiguration { ... }
```

Use cases:

- Shared company libraries (auth, logging, audit) across microservices
- Wrapping third-party SDKs (payment gateway, SMS provider) for easy integration

Q34 ⭐ [FREQUENTLY ASKED]: In Spring, what is the difference between @Mock and @MockBean annotations?

@Mock (Mockito):

- Creates a Mockito mock outside the

Spring context

- Used in pure unit tests with @ExtendWith(MockitoExtension.class)
- Faster — no Spring context initialization

@MockBean (Spring Boot Test):

- Creates a mock and registers it as a bean in the Spring Application Context
- Replaces the real bean in the context for integration tests
- Used with @SpringBootTest or slice tests like @WebMvcTest

```
@WebMvcTest
class ControllerTest {
  @MockBean UserService
userService; // replaces real bean
  @Autowired MockMvc mockMvc;
}
```

Q35 ⭐ [FREQUENTLY ASKED]: Explain the role of the @Async annotation in Spring Framework.

- @Async marks a method to be executed in a separate thread (asynchronously)
- Must enable with @EnableAsync on a @Configuration class
- Returns void, Future<T>, CompletableFuture<T>, or ListenableFuture<T>

```
@Async
public CompletableFuture<String>
processImage(File file) {
  // runs in background thread
  return
CompletableFuture.completedFuture(resu
lt);
}
```

- Default executor is a SimpleAsyncTaskExecutor — configure a ThreadPoolTaskExecutor in production
- Important: @Async doesn't work when called within the same class (proxy limitation)
- Useful for: email sending, file processing, API calls, report generation

Q36: How does Spring handle scheduling and task execution?

- Enable with @EnableScheduling on a

@Configuration class

- @Scheduled on methods with fixedRate, fixedDelay, or cron expressions

```
@Scheduled(cron = "0 0 2 * * ?")
public void dailyCleanup() { ... }

@Scheduled(fixedRate = 5000) // every 5 seconds
public void pollFeed() { ... }
```

- TaskScheduler interface for programmatic scheduling
- @EnableAsync + ThreadPoolTaskScheduler for concurrent task execution
- In distributed systems, use ShedLock or Quartz Scheduler to prevent duplicate execution

Q37: Discuss the benefits and considerations of using externalized configuration in Spring Boot.

Benefits:

- Same artifact runs in dev/test/prod — only config changes (12-factor app principle)
- Secrets stay out of source code
- Configuration can be changed without recompilation or redeployment (with config server)

Considerations:

- Priority order: Command-line args > OS env vars > application.properties > defaults
- Use Spring Cloud Config Server for centralized config in microservices
- Encrypt sensitive properties using Jasypt or Spring Cloud Config encryption
- Validate configuration at startup with @Validated on @ConfigurationProperties

Q38 ⭐ [FREQUENTLY ASKED]: What is method security in Spring, and how can it be applied at the service layer?

- Method security enables fine-grained access control at the method level
- Enable with @EnableMethodSecurity

(Spring Security 6+)

- @PreAuthorize: Checks before method execution using SpEL expressions
- @PostAuthorize: Checks after method execution (can use return value)
- @Secured: Simpler role-based check; @RolesAllowed: JSR-250 equivalent

```
@PreAuthorize("hasRole('ADMIN') or
#userId ==
authentication.principal.id")
public User getUser(Long userId) { ...
}
@PostAuthorize("returnObject.ownerId
== authentication.name")
public Document getDocument(Long id) {
... }
```

Q39 ⭐ [FREQUENTLY ASKED]: What are alternatives of @Autowired?

- Constructor Injection (RECOMMENDED): Most preferred; ensures required dependencies; immutable

```
public class UserService {
  private final UserRepository repo;
  public UserService(UserRepository
repo) { this.repo = repo; }
}
```

- @Inject (JSR-330): Standard Java annotation; same behavior as @Autowired
- @Resource (JSR-250): Injects by name by default; part of Jakarta EE
- @Value: Injects property values from configuration
- ApplicationContext.getBean(): Programmatic lookup — avoid; breaks IoC
- Best practice: Prefer constructor injection — makes testing easier and dependencies explicit

Q40 ⭐ [FREQUENTLY ASKED]: What is the difference between JUnit 4 and JUnit 5, and why might you choose one over the other?

JUnit 4 vs JUnit 5 comparison:

- Architecture: JUnit 4 is monolithic; JUnit 5 has three modules (Platform, Jupiter, Vintage)
- Annotations: @Before → @BeforeEach; @After → @AfterEach;

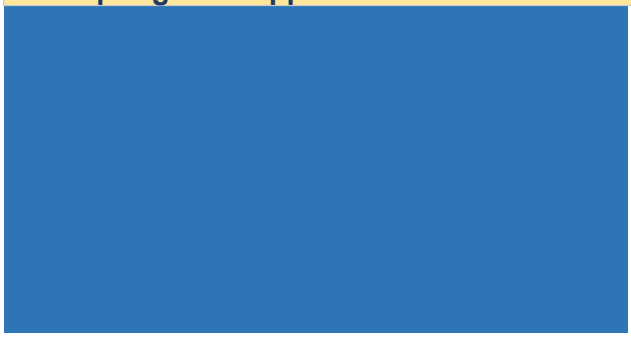
@BeforeClass → @BeforeAll

- Extension model: JUnit 4 uses @RunWith; JUnit 5 uses @ExtendWith (composable)
- Parameter injection: JUnit 5 supports method parameter injection (e.g., TestInfo, TestReporter)
- Nested tests: JUnit 5 supports @Nested for organized test classes
- Dynamic tests: JUnit 5 @TestFactory for programmatic test generation
- Java version: JUnit 5 requires Java 8+; JUnit 4 supports Java 5+
- Choose JUnit 5 for new projects — it is more modern, extensible, and actively maintained

Q41: Can you use both application.yml and application.properties in a Spring Boot project? If so, how are they prioritized?

- Yes, both can coexist in the same project
- application.properties takes priority over application.yml when both are present
- Profile-specific files (application-dev.properties) take priority over the base files
- If the same key exists in both, application.properties value wins
- Best practice: Use one format consistently to avoid confusion
- YAML supports nested structure and lists more naturally than flat properties
- Spring Boot reads both from classpath: root at startup

Q42 ⭐ [FREQUENTLY ASKED]: How do you configure and connect multiple databases in a Spring Boot application?



- Define separate DataSource beans annotated with @Primary (for the default)
- Create separate EntityManagerFactory and TransactionManager for each DB
- Use @EnableJpaRepositories with entityManagerFactoryRef and transactionManagerRef
- Place entity classes and repositories in

different packages per database

```
@Configuration
@EnableJpaRepositories(basePackages="com.app.orders",
  entityManagerFactoryRef="ordersEMF",
transactionManagerRef="ordersTM")
public class OrdersDbConfig { ... }
```

- Bind each datasource config from separate property prefixes:

```
spring.datasource.orders.url=...
spring.datasource.users.url=...
```

Q43 ⭐ [FREQUENTLY ASKED]: What is the difference between returning a ResponseEntity vs directly returning an object in a REST API?

Direct Object Return:

- Simpler code; Spring auto-sets 200 OK status
- No control over HTTP status code, headers, or body for edge cases

ResponseEntity<T>:

- Full control over status code, headers, and response body
- Can return different status codes: 201 Created, 204 No Content, 404 Not Found

```
return
ResponseEntity.created(URI.create("/users/" + id)).body(user);

return
ResponseEntity.noContent().build(); // 204
```

- Best practice: Use direct return for simple GETs; ResponseEntity for create/update/delete

Q44 ⭐ [FREQUENTLY ASKED]: You are tasked with designing RESTful endpoints for a complex product inventory system. What best practices would you follow?

- Use nouns for resources: /products, /products/{id}/variants
- HTTP methods: GET (read), POST (create), PUT/PATCH (update), DELETE (delete)
- Versioning: /api/v1/products — prefix or Accept header versioning
- Pagination: ?page=0&size=20&sort=name,asc for list endpoints
- Filtering: ?

category=electronics&minPrice=100 as query params

- Consistent error responses: RFC 7807 ProblemDetail format
- HATEOAS: Include related links in responses for discoverability
- Idempotency: PUT/DELETE must be idempotent; use idempotency-key for POST
- Rate limiting, authentication (JWT), and API documentation (OpenAPI 3.0)

Q45 ⭐ [FREQUENTLY ASKED]: What is the Spring Boot @Profile annotation used for?

- @Profile activates beans only when a specific profile is active
- Use case: Different DataSource configs for dev (H2) vs prod (PostgreSQL)

```
@Profile("dev")
@Bean public DataSource
devDataSource() { return new
EmbeddedDatabase...; }
@Profile("prod")
@Bean public DataSource
prodDataSource() { return
hikariDataSource(); }
```

- Activate profile: spring.profiles.active=prod in properties or -Dspring.profiles.active=prod
- Use @Profile on @Configuration, @Component, or @Bean methods
- Combine: @Profile({"dev","test"}) — active if either profile is active
- Profile-specific property files: application-prod.yml auto-loaded when prod is active

Q46: Describe how you would set up integration tests for a Spring Boot application that interacts with an external API.

- Use @SpringBootTest to load full application context
- Use WireMock to stub the external API responses in tests

```
@AutoConfigureWireMock(port = 8089)
@SpringBootTest
class ExternalApiIntegrationTest { ...
}
```

- Configure the API base URL to point to WireMock server in test

application.properties

- Use Testcontainers to spin up real databases alongside WireMock
- Test happy path, error responses (4xx/5xx), timeouts, and retry scenarios
- Use RestTemplate/WebClient interceptors to log requests/responses in tests

Q47 ⭐ [FREQUENTLY ASKED]: How does the @Qualifier annotation work in Spring for managing dependencies?

- @Qualifier disambiguates when multiple beans of the same type exist

```
@Autowired @Qualifier("emailNotifier")
private Notifier notifier;
```

- The qualifier value matches the bean name or the value in @Qualifier on the @Bean definition
- Used with constructor injection:

```
public
UserService(@Qualifier("primaryDB")
DataSource ds) { ... }
```

- Can create custom qualifier annotations to group beans by category
- @Primary is an alternative — marks the default bean to inject when no qualifier is specified

Q48: Your project needs to integrate a third-party library. How would you proceed to create a Starter for this library?

- Create two Maven modules: my-library-spring-boot-autoconfigure and my-library-spring-boot-starter
- Starter pom.xml depends on both autoconfigure module and the third-party library
- In autoconfigure: Create @AutoConfiguration class with @ConditionalOnClass(LibraryClass.class)
- Define @ConfigurationProperties for the library's settings
- Auto-create the library bean using the properties
- Register the auto-configuration in META-INF/spring/org.springframework.boot.au

toconfigure.AutoConfiguration.imports

- Write tests using @SpringBootTest to verify auto-configuration behavior

Q49: Can you name a few common starters used in Spring Boot applications?

- spring-boot-starter-web — Spring MVC, Tomcat, Jackson for REST APIs
- spring-boot-starter-data-jpa — Hibernate, Spring Data JPA, JDBC
- spring-boot-starter-security — Spring Security for auth/authz
- spring-boot-starter-test — JUnit 5, Mockito, Spring Test utilities
- spring-boot-starter-actuator — production monitoring endpoints
- spring-boot-starter-data-redis — Redis integration via Lettuce or Jedis
- spring-boot-starter-amqp — RabbitMQ messaging via Spring AMQP
- spring-boot-starter-webflux — reactive web with Project Reactor/Netty
- spring-boot-starter-validation — Bean Validation with Hibernate Validator

Q50: What is the purpose of having a spring-boot-starter-parent?

- Provides default dependency management — versions aligned for compatibility
- Inherits from spring-boot-dependencies BOM (Bill of Materials)
- Pre-configured Maven plugins: compiler, surefire, failsafe, spring-boot plugin
- Default resource filtering for application.properties and application.yml
- Sets Java version, encoding (UTF-8), and other project defaults
- Eliminates version conflicts — just add dependencies without specifying versions

```
<parent>

<groupId>org.springframework.boot</gro
upId>
  <artifactId>spring-boot-starter-
parent</artifactId>
```

```
  <version>3.2.0</version>
</parent>
```

Q51 ⭐ [FREQUENTLY ASKED]: How to handle asynchronous operations in Spring Boot?

- @Async + @EnableAsync: Offload method to background thread pool
- CompletableFuture: Chain async operations, combine results
- Reactive (WebFlux): Mono/Flux for fully non-blocking async streams
- Message Queues: RabbitMQ, Kafka for decoupled async processing
- @Scheduled: Time-triggered async background tasks
- Spring Integration: Complex async workflow orchestration

```
@Async
public CompletableFuture<Report>
generateReport(Long id) {
  return
CompletableFuture.supplyAsync(() ->
reportService.generate(id));
}
```

- Always configure a ThreadPoolTaskExecutor with proper pool sizes and queue capacity

Q52: You need to implement a feature that processes heavy image files asynchronously. How would you set up and manage these operations in Spring Boot?

- Use @Async with a dedicated ThreadPoolTaskExecutor for image processing
- Return CompletableFuture<String> (URL of processed image) from the async method
- Configure thread pool: corePoolSize=4, maxPoolSize=10, queueCapacity=100

```
@Bean public TaskExecutor
imageExecutor() {
  ThreadPoolTaskExecutor exec = new
ThreadPoolTaskExecutor();
  exec.setCorePoolSize(4);
exec.setMaxPoolSize(10);
  exec.setThreadNamePrefix("image-");
return exec;
}
```

- Store upload → queue processing job → return job ID immediately to client
- Client polls /jobs/{id}/status or use

WebSocket/SSE for progress updates

- Use S3 or Azure Blob Storage for processed image storage
- Handle errors with CompletableFuture.exceptionally() and DLQ for failed jobs

Q53: How do you handle session management in Spring Boot?

- Default: Servlet container session (stored in-memory; not distributed)
- Spring Session: Externalizes session to Redis, JDBC, or Hazelcast
- Add spring-session-data-redis + @EnableRedisHttpSession for Redis-backed sessions

```
spring.session.store-type=redis
spring.session.redis.flush-
mode=on_save
```

- JWT-based: Stateless; no server-side session; token carries claims
- Configure session timeout: server.servlet.session.timeout=30m
- Secure cookies: server.servlet.session.cookie.secure=true; httponly=true

Q54: How would you configure session clustering in a Spring Boot application?

- Use Spring Session with Redis as the session store
- All application instances connect to the same Redis cluster
- Session is serialized to Redis on each request

```
@EnableRedisHttpSession(maxInactiveInt
ervalInSeconds = 1800)
public class SessionConfig { }
```

- Sticky sessions (load balancer): Route user to same instance — less robust
- Redis Cluster or Sentinel for HA of the session store itself
- Test failover by killing one instance — user session should persist on others

Q55: Your application is experiencing session loss when deployed across multiple servers. What strategy would you implement?

- Root cause: In-memory sessions are

not shared across instances

- Immediate fix: Enable Spring Session with Redis — sessions stored externally
- Add spring-session-data-redis dependency and @EnableRedisHttpSession
- Ensure all instances connect to the same Redis instance/cluster
- Alternative: Switch to JWT-based stateless authentication — no server sessions
- If using JWT, store refresh tokens in Redis for revocation capability
- Update load balancer to NOT use sticky sessions (no longer needed with Redis)

Q56 ⭐ [FREQUENTLY ASKED]: What is the role of @SpringBootTest annotation?

- @SpringBootTest loads the full Spring Application Context for integration testing
- Provides a real application context — all beans, configurations, and auto-configurations active
- webEnvironment options: MOCK (default), RANDOM_PORT, DEFINED_PORT, NONE

```
@SpringBootTest(webEnvironment =
SpringBootTest.WebEnvironment.RANDOM_P
ORT)
class OrderApiIntegrationTest { ... }
```

- Use @TestPropertySource or properties attribute to override config in tests
- Slower than slice tests (@WebMvcTest, @DataJpaTest) — use only for full integration tests
- Use @DirtiesContext if a test modifies the application context

Q57 ⭐ [FREQUENTLY ASKED]: How do you write unit tests for Spring Boot controllers?

- Use @WebMvcTest(ControllerClass.class) — loads only web layer (no service/repo)
- Inject MockMvc to perform HTTP requests without starting a real server

- Use @MockBean to mock service layer dependencies

```
@WebMvcTest(UserController.class)
class UserControllerTest {
  @Autowired MockMvc mockMvc;
  @MockBean UserService userService;

  @Test void shouldReturnUser() throws
Exception {

given(userService.findById(1L)).willRe
turn(new User("John"));
    mockMvc.perform(get("/users/1"))
      .andExpect(status().isOk())

.andExpect(jsonPath("$.name").value("J
ohn"));
  }
}
```

- Test HTTP status, response body (jsonPath), headers, and content type
- Use MockMvcResultMatchers for assertions

Q58 ⭐ [FREQUENTLY ASKED]: Can you list some of the endpoints provided by Spring Boot Actuator?

- /actuator/health — application health status (UP/DOWN) with component details
- /actuator/info — application info (version, description from properties)
- /actuator/metrics — JVM, HTTP, datasource, cache metrics
- /actuator/metrics/{name} — specific metric value (e.g., jvm.memory.used)
- /actuator/env — environment properties and property sources
- /actuator/beans — all Spring beans in the context
- /actuator/mappings — all HTTP request mappings
- /actuator/loggers — view and change log levels at runtime
- /actuator/httptrace — recent HTTP request/response traces
- /actuator/threaddump, /actuator/heapdump — JVM diagnostics

Q59: How do you customize Actuator endpoints?

- Enable/disable specific endpoints:

management.endpoints.web.exposure.include=health,info,metrics

- Change base path: management.endpoints.web.base-path=/admin
- Custom health indicator: implement HealthIndicator interface

```
@Component
public class DatabaseHealthIndicator
implements HealthIndicator {
  @Override public Health health() {
    return
Health.up().withDetail("version",
"8.0").build();
  }
}
```

- Custom info contributor: implement InfoContributor
- Secure actuator endpoints: configure SecurityFilterChain to restrict /actuator/**
- Custom metrics: inject MeterRegistry and register Gauge, Counter, Timer

Q60: How can Actuator be used for application monitoring and management?

- Health endpoint: load balancer/K8s readiness and liveness probes
- Metrics endpoint: feed data to Prometheus/Grafana for dashboards
- Loggers endpoint: change log level in production without restart (critical for debugging)
- Env endpoint: verify which configuration properties are active
- Heapdump: capture heap for OOM analysis
- Integrate with Micrometer for vendor-neutral metrics export (Prometheus, Datadog, CloudWatch)

```
management.metrics.export.prometheus.enabled=true
```

Q61 ⭐ [FREQUENTLY ASKED]: What is the order of precedence in Spring Boot configuration?

From highest to lowest priority:

- 1. Command-line arguments (--server.port=9090)
- 2. SPRING_APPLICATION_JSON

(inline JSON in env variable)

- 3. OS environment variables (SERVER_PORT=9090)
- 4. System properties (-Dserver.port=9090)
- 5. Profile-specific files (application-prod.yml)
- 6. Application files (application.yml / application.properties)
- 7. @PropertySource annotations
- 8. Default properties (SpringApplication.setDefaultProperties)
- Spring Cloud Config Server properties override local files when properly ordered

Q62: Can you deploy a Spring Boot application as a traditional WAR file to an external server?

- Yes — change packaging to war in pom.xml
- Extend SpringBootServletInitializer and override configure() method
- Mark spring-boot-starter-tomcat as provided scope (server provides it)
- Build: mvn package → creates app.war
- Drop WAR into Tomcat's webapps/ folder or deploy via Tomcat Manager
- Spring Boot will still bootstrap correctly using the Servlet container's lifecycle
- Context path defaults to WAR file name: /app — configure in server.xml or context.xml

Q63: You are transitioning an existing application from properties to YAML. Describe the steps and considerations.

- Rename application.properties to application.yml
- Convert flat key=value to nested YAML structure:

```
# Before:
spring.datasource.url=jdbc:mysql://loc
alhost/db
# After:
spring:
  datasource:
    url: jdbc:mysql://localhost/db
```

- YAML uses 2-space indentation — tabs

are NOT allowed

- Lists in YAML use - prefix instead of comma-separated values
- Multiple profiles: use --- separator to define profiles in one file
- Validate with a YAML linter before deploying
- Remove old properties files to avoid confusion with precedence

Q64 ⭐ [FREQUENTLY ASKED]: How does Spring Boot support data validation?

- Integrates with Jakarta Bean Validation (JSR-380) via Hibernate Validator
- Add spring-boot-starter-validation dependency
- Annotations: @NotNull, @NotBlank, @Size, @Min, @Max, @Email, @Pattern, @Positive
- Validate @RequestBody with @Valid in controllers
- Validate @ConfigurationProperties with @Validated for startup validation

```
@ConfigurationProperties("app")
@Validated
public class AppProperties {
  @NotBlank private String apiKey;
}
```

- ValidationAutoConfiguration auto-registers the Validator bean

Q65: Can you use custom validators in Spring Boot? How?

- Yes — implement the ConstraintValidator interface
- Step 1: Create custom annotation:

```
@Constraint(validatedBy =
PhoneValidator.class)
@Target({ElementType.FIELD})
public @interface ValidPhone { String
message() default "Invalid phone"; ...
}
```

- Step 2: Implement ConstraintValidator<ValidPhone, String>:

```
public class PhoneValidator implements
ConstraintValidator<ValidPhone,
String> {
  public boolean isValid(String value,
ConstraintValidatorContext ctx) {
    return value != null &&
```

```
value.matches("\\d{10}");
  }
}
```

- Apply @ValidPhone on any field — Spring auto-discovers it via classpath scanning

Q66: You need to implement complex validation rules that involve multiple fields of a form. Describe your approach using Spring Boot.

- Use class-level constraints for cross-field validation
- Create annotation targeting ElementType.TYPE

```
@PasswordMatch
public class RegistrationRequest {
  private String password;
  private String confirmPassword;
}
```

- PasswordMatchValidator checks both fields in isValid()
- Alternative: Override validate() in a Spring Validator implementation
- Use validation groups for different validation scenarios (Create vs Update)

```
@Validated(CreateGroup.class)
@RequestBody UserRequest req
```

- Custom error messages defined in ValidationMessages.properties for i18n support

Q67 ⭐ [FREQUENTLY ASKED]: In JUnit testing, what are the annotations @Before, @After, @BeforeAll?

JUnit 4 → JUnit 5 equivalents:

- @Before (@BeforeEach in JUnit 5): Runs before EACH test method — setup per test
- @After (@AfterEach in JUnit 5): Runs after EACH test method — teardown per test
- @BeforeClass (@BeforeAll in JUnit 5): Runs ONCE before all tests in class — static method
- @AfterClass (@AfterAll in JUnit 5): Runs ONCE after all tests in class — static method
- @BeforeAll/@AfterAll must be static (unless class uses @TestInstance(Lifecycle.PER_CLASS))

Use cases:

- @BeforeEach: Initialize mocks, set up test data, open connections
- @BeforeAll: Start expensive resources (database, WireMock server) once for all tests

Q68: How would you use these annotations in a practical test case?

```
@SpringBootTest
class ProductServiceTest {
  @BeforeAll
  static void startServer()
{ wireMockServer.start(); }

  @BeforeEach
  void setUp()
{ productRepo.deleteAll();
productRepo.save(new Product("Test"));
}

  @Test
  void shouldFindProductById() {
    Product p =
productService.findById(1L);

assertThat(p.getName()).isEqualTo("Tes
t");
  }

  @AfterEach
  void cleanUp()
{ productRepo.deleteAll(); }

  @AfterAll
  static void stopServer()
{ wireMockServer.stop(); }
}
```

Q69 ⭐ [FREQUENTLY ASKED]: What are the HTTP methods?

- GET: Retrieve a resource; idempotent; safe; no body
- POST: Create a new resource; not idempotent; has body
- PUT: Replace an entire resource; idempotent
- PATCH: Partially update a resource; not necessarily idempotent
- DELETE: Remove a resource; idempotent
- HEAD: Same as GET but returns only headers (no body)
- OPTIONS: Describe communication options for a resource (used in CORS

preflight)

- TRACE: Loop-back test for diagnostics — rarely used in APIs

Q70 ⭐ [FREQUENTLY ASKED]: Can you explain when to use each HTTP method in the context of RESTful APIs?

- GET /products — list all products (no side effects)
- GET /products/{id} — get a specific product
- POST /products — create a new product; returns 201 + Location header
- PUT /products/{id} — replace entire product (client sends full representation)
- PATCH /products/{id} — update specific fields (client sends partial data)
- DELETE /products/{id} — remove product; returns 204 No Content
- OPTIONS /products — used by browser for CORS preflight request
- Rule: GET/HEAD/OPTIONS are safe; GET/PUT/DELETE/HEAD/OPTIONS are idempotent

Q71 ⭐ [FREQUENTLY ASKED]: How can you implement authentication and authorization in Spring Boot?

Authentication (Who are you?):

- Form login, HTTP Basic, JWT, OAuth2/OIDC via Spring Security
- JWT: Client sends token in Authorization: Bearer header; Spring validates it

```
@Bean SecurityFilterChain
filterChain(HttpSecurity http) {
  return http.oauth2ResourceServer(o
->
o.jwt(Customizer.withDefaults())).buil
d();
}
```

Authorization (What can you do?):

- URL-based: .requestMatchers("/admin/**").hasRole("ADMIN")
- Method-based: @PreAuthorize("hasRole('ADMIN')") at service layer
- OAuth2: Use Keycloak or Auth0 as

identity provider; Spring Boot acts as resource server

Q72: How does the choice of YAML over properties files affect the application's performance?

- Negligible performance difference — both are read once at startup
- YAML parsing is slightly slower than properties parsing due to more complex grammar
- At runtime, both are stored as the same PropertySource — no difference in access speed
- YAML is generally preferred for readability and hierarchy in complex configurations
- Performance impact is measured in milliseconds — not significant for most applications
- Startup time difference is negligible even with large configuration files

Q73 ⭐ [FREQUENTLY ASKED]: You have a complex query that runs slowly in your Spring Boot application. How would you optimize it?

- Enable SQL logging: spring.jpa.show-sql=true + Hibernate statistics to measure query time
- Check execution plan (EXPLAIN ANALYZE) in the database
- Add database indexes on columns used in WHERE, JOIN, ORDER BY clauses
- Fix N+1 problem: Use JOIN FETCH or @EntityGraph in JPQL/Criteria API
- Use @Query with native SQL for complex queries (bypass HQL overhead)
- Pagination: Never fetch all records; use Pageable parameter
- Projections: Select only needed columns (interface/DTO projections in Spring Data)
- Query result caching: @Cacheable for read-heavy infrequently-changed data
- Read replicas: Route SELECT queries to replica via AbstractRoutingDataSource

Q74: What are Spring Boot DevTools?

- spring-boot-devtools is a development-time convenience module
- Automatic restart: Detects classpath changes and restarts app (much faster than cold start)
- LiveReload: Triggers browser refresh when static resources change
- Relaxed property defaults: e.g., disables template caching, enables debug logging
- Remote DevTools: Supports remote update triggering via HTTP tunnel
- H2 console auto-enabled in development
- DevTools is automatically disabled when running a packaged JAR (production safe)

Q75: What should be considered when using Spring Boot DevTools in production environments?

- DevTools auto-disables when app runs as fully packaged JAR — production is safe by default
- If somehow included, it would cause: frequent restarts on any classpath change (destructive)
- LiveReload server (port 35729) would be exposed — security concern
- Best practice: Mark devtools as optional in pom.xml or use developmentOnly scope in Gradle

```
<dependency>

<groupId>org.springframework.boot</gro
upId>
  <artifactId>spring-boot-
devtools</artifactId>
  <optional>true</optional>
</dependency>
```

- CI/CD pipelines should use mvn package (not spring-boot:run) ensuring devtools excluded

Q76: You're developing an application that requires frequent changes and immediate feedback. How can DevTools assist?

- Automatic restart kicks in seconds after

saving — no manual restart needed

- LiveReload integration: Browser automatically refreshes when Thymeleaf templates or CSS change
- Template caching disabled: Thymeleaf/FreeMarker changes visible immediately without restart
- H2 console available at /h2-console for instant database inspection
- Remote DevTools: If working with a remote dev environment, trigger restarts over HTTP
- Combine with IDE's hot swap (DCEVM) for method body changes without restart
- Result: Inner development loop goes from 30-60 seconds to 2-5 seconds

Step 13 – Spring Boot Level V (Expert)

This section covers 30 expert-level Spring Boot questions requiring hands-on experience and architectural thinking.

Q1 ⭐ [FREQUENTLY ASKED]: You might have created REST API from scratch. Tell me 5 REST API annotations.

- @RestController — Combines @Controller + @ResponseBody; marks class as REST endpoint
- @RequestMapping — Maps HTTP requests to handler methods (class or method level)
- @GetMapping / @PostMapping / @PutMapping / @DeleteMapping / @PatchMapping — HTTP method shortcuts
- @PathVariable — Extracts value from URL path: GET /users/{id}
- @RequestBody — Deserializes JSON request body into Java object
- Bonus: @RequestParam (query param), @ResponseStatus, @ExceptionHandler

```
@RestController
@RequestMapping("/api/v1/users")
public class UserController {
  @GetMapping("/{id}")
  public User getUser(@PathVariable
Long id) { ... }
}
```

Q2 ⭐ [FREQUENTLY ASKED]: Have you encountered any real-world bugs due to incorrect choice between PUT and POST? How did you resolve it?

Common Scenario:

- Using POST for an update operation — multiple retries on network failure caused duplicate records
- Root Cause: POST is not idempotent; retry creates multiple resources

Resolution:

- Changed to PUT (idempotent) — retries are safe as the same state results
- Added idempotency keys for POST operations that must not be duplicated (e.g., payments)

- Client sends Idempotency-Key header; server caches response by key to detect replays
- Lesson: Always consider idempotency when choosing HTTP method, especially for retries and payment flows

Q3 ⭐ [FREQUENTLY ASKED]: If you have multiple beans of the same type, how would you handle conflicts using @Autowired and @Qualifier?

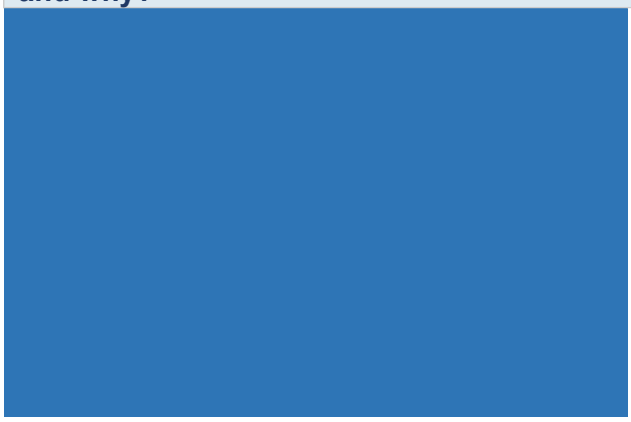
- Spring throws NoUniqueBeanDefinitionException when multiple beans match without disambiguation
- Use @Qualifier("beanName") at injection point to specify which bean to inject
- Use @Primary on one bean to make it the default
- Avoid mixing @Primary and @Qualifier inconsistently — it causes confusion

Potential Issues:

- If @Qualifier name doesn't match any bean name → NoSuchBeanDefinitionException
- Using @Primary + @Qualifier on different beans for same injection → Qualifier wins
- Best: Use constructor injection with @Qualifier for explicit, testable wiring

```
@Autowired
public Service(@Qualifier("fastCache")
CacheManager cm) { this.cm = cm; }
```

Q4: If your application needs to handle file uploads, which controller would you prefer and why?



- Use @RestController with MultipartFile parameter for REST APIs (JSON + file upload)
- Use @Controller (not Rest) only for traditional form-based file upload with view rendering

```
@PostMapping("/upload")
public ResponseEntity<String>
upload(@RequestParam MultipartFile
file) {
  // process file
  return ResponseEntity.ok("Uploaded:
" + file.getOriginalFilename());
```

```
}
```

@RestController Complications:

- Mixing multipart/form-data with application/json in one endpoint requires custom parsing
- Large file uploads need careful config: spring.servlet.multipart.max-file-size=50MB
- For very large files, use streaming (InputStream) instead of MultipartFile to avoid OOM

Q5 ⭐ [FREQUENTLY ASKED]: Are you using cache? If yes, in which scenario? When are you removing/refreshing data from cache?

Yes — common caching scenarios:

- Product catalog, pricing, and configuration data (rarely changes, read frequently)
- User session data and authentication tokens
- Expensive computation results (report data, aggregations)

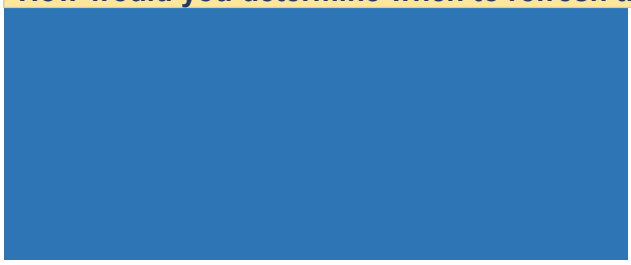
Cache Eviction Strategies:

- TTL (Time-To-Live): Cache expires automatically after set duration — good for catalog
- @CacheEvict: Remove specific entry on update/delete operations
- @CachePut: Always update cache on write

```
@CacheEvict(value = "products", key =
"#id")
public void updateProduct(Long id,
Product p) { ... }
```

- Event-driven: Listen for DB change events and invalidate cache entries

Q6 ⭐ [FREQUENTLY ASKED]: Assume your cache is stale due to frequent updates. How would you determine when to refresh the cache?



- Monitor cache hit rate — low hit rate with high miss rate indicates stale/over-evicted cache
- Implement cache versioning: increment version key when data changes; old entries become unreachable
- Use write-through caching: update

cache and DB atomically on every write (@CachePut)

- Event-driven invalidation: Publish domain events on DB updates; subscribers evict affected keys
- Redis keyspace notifications: Subscribe to key expiry events for cache warm-up
- Implement near-cache pattern: L1 local Caffeine + L2 Redis; invalidate L1 via pub/sub
- Set aggressive TTL with background refresh (Caffeine's refreshAfterWrite) for critical data

Q7 ⭐ [FREQUENTLY ASKED]: How are you verifying your changes once your changes are deployed to production?

- Smoke tests: Run automated API tests against production endpoints immediately post-deploy
- Canary deployment: Route 5-10% traffic to new version; monitor error rate and latency
- Actuator health: Check /actuator/health — verify all components (DB, Redis, queues) are UP
- Application metrics: Monitor in Grafana/Datadog — error rates, response times, throughput
- Log monitoring: Watch for ERROR/WARN spikes in Kibana/CloudWatch Logs
- Feature flags: Gradually enable new features and monitor per-flag metrics
- Rollback criteria: Define thresholds (e.g., error rate >1%) that trigger automatic rollback

Q8 ⭐ [FREQUENTLY ASKED]: If you discover a critical bug shortly after deployment, what processes do you have in place to roll back quickly?

- Blue-Green: Switch load balancer back to blue (old) environment instantly
- Kubernetes: kubectl rollout undo deployment/app — rolls back to previous ReplicaSet
- Feature flags: Disable the flag exposing buggy feature — no deployment

needed

- Database: Use expand-contract pattern — new schema additions are backward compatible
- Maintain rollback runbook with specific commands and verification steps
- Keep previous artifact (JAR/Docker image) tagged and accessible for fast redeploy
- Automated rollback: CI/CD monitors post-deploy metrics and triggers rollback if threshold exceeded

Q9: Where are you storing your artifacts?

- JAR/WAR: Nexus Repository, Artifactory, AWS CodeArtifact, GitHub Packages
- Docker Images: AWS ECR, Docker Hub, Azure Container Registry, Google Artifact Registry
- Helm Charts: Helm Chart Museum or OCI-based registry (ECR, ACR)
- Infrastructure as Code: Git repository (version controlled alongside application code)
- CI/CD Pipeline publishes artifact on every successful build — tagged with version + git SHA
- Semantic versioning: MAJOR.MINOR.PATCH + SNAPSHOT for dev builds, RELEASE for prod

Q10: If your artifact storage becomes inaccessible, what contingency plans do you have to ensure your deployment process can continue?

- Mirror registry in secondary region — failover DNS to backup registry
- Cache Docker layers in local registry mirror (Docker registry proxy) on build agents
- Keep last N deployed images in cluster (K8s imagePullPolicy=Never if image already present)
- Store critical artifacts in S3/Blob storage as backup alongside primary registry
- CI/CD pipelines should have timeout and fallback paths documented

- Periodic DR drills to validate artifact recovery procedures

Q11 ⭐ [FREQUENTLY ASKED]: What is the CAP theorem?

CAP Theorem: A distributed system can only guarantee 2 of these 3 properties:

- Consistency (C): Every read receives the most recent write or an error
- Availability (A): Every request receives a response (not necessarily latest data)
- Partition Tolerance (P): System continues operating despite network partitions

In practice — partition tolerance is non-negotiable in distributed systems:

- CP Systems: Strong consistency; may become unavailable during partition (e.g., ZooKeeper, HBase)
- AP Systems: Always available; may return stale data during partition (e.g., Cassandra, DynamoDB)
- CA systems don't exist in distributed computing (can't avoid network partitions)

Q12 ⭐ [FREQUENTLY ASKED]: Can you provide a real-world scenario where you had to prioritize between consistency, availability, and partition tolerance?

Scenario: E-commerce Shopping Cart

- Chose AP (availability + partition tolerance) for shopping cart — users must always add items
- Cart data uses Cassandra (AP system) — slight inconsistency acceptable if cart reads stale
- On the other hand, inventory and payments use PostgreSQL with strong consistency (CP)
- Reasoning: It's better to show a slightly stale cart than block checkout entirely
- Inventory deduction uses optimistic locking + eventual consistency with compensation
- Lesson: Different parts of the same system can have different CAP choices

Q13 ⭐ [FREQUENTLY ASKED]: Can you talk about a time when a misconfiguration in the properties file caused an issue in production?

Common Real-World Scenario:

- Incorrect database connection pool size (maximum-pool-size=2) caused connection timeouts under load
- spring.jpa.hibernate.ddl-auto=create-drop in production wiped the database on restart
- Logging level set to TRACE in production caused disk full and application crash

Prevention:

- Use @Validated on @ConfigurationProperties with constraints for startup validation
- Code review checklist for production property files
- Spring Cloud Config with audit trail for configuration changes
- Environment-specific property validation in CI/CD pipeline before deployment

Q14 ⭐ [FREQUENTLY ASKED]: Consider a scenario where a transaction spans multiple service calls. How would you decide the appropriate propagation level for each service call?

- REQUIRED (default): Join existing transaction; good for core business operations
- REQUIRES_NEW: Start a new independent transaction; use for audit logging, notifications — must complete regardless of outer tx
- NESTED: Savepoint within outer tx; can rollback inner without rolling back outer
- SUPPORTS: Run in transaction if exists; otherwise run without — good for reads
- NOT_SUPPORTED: Always run without transaction — good for reporting, caching

Pitfalls:

- Self-invocation bypasses proxy → propagation not applied
- REQUIRES_NEW suspends outer tx → DB lock contention if tables overlap

- Incorrect propagation in payment service → double charges or missed rollback

Q15 ⭐ [FREQUENTLY ASKED]: What are the best practices for designing a custom exception handling framework in Spring Boot?

- @ControllerAdvice with @ExceptionHandler — global exception handling in one place
- Return RFC 7807 ProblemDetail or custom ErrorResponse with: status, message, timestamp, traceId
- Define exception hierarchy: BusinessException > (OrderException, PaymentException)
- Map exception types to HTTP status codes in a consistent manner

```
@ExceptionHandler(EntityNotFoundExcept
ion.class)
public ResponseEntity<ProblemDetail>
handle(EntityNotFoundException ex) {
  ProblemDetail pd =
ProblemDetail.forStatusAndDetail(HttpS
tatus.NOT_FOUND, ex.getMessage());
  return
ResponseEntity.status(404).body(pd);
}
```

- Never expose stack traces in production API responses — log them internally
- Use traceId (from MDC/Micrometer) in response for correlation with logs

Q16: How do you manage configuration changes for your Dockerized Spring Boot application across different environments?

- Environment variables: Pass config via docker run -e or K8s ConfigMaps/Secrets
- Spring profiles: SPRING_PROFILES_ACTIVE=prod env var selects the right config
- Spring Cloud Config Server: Central config management; apps fetch config at startup
- Kubernetes Secrets for sensitive values (passwords, API keys) → mounted as env vars
- External Secrets Operator: Sync secrets from AWS Secrets Manager into K8s Secrets

- Never bake environment-specific config into Docker image — keep it external
- Config change without restart: Spring Cloud Bus + /actuator/refresh for hot reload

Q17: As part of a move to microservices, you are containerizing your Spring Boot applications. What steps would you follow?

- Multi-stage Dockerfile: Build stage (Maven) + runtime stage (JRE-only image)
- Use minimal base image: eclipse-temurin:21-jre-alpine to reduce attack surface
- Run as non-root user in container for security
- Set JVM flags for container awareness: -XX:MaxRAMPercentage=75.0
- Configure health checks: HEALTHCHECK CMD curl -f http://localhost:8080/actuator/health
- Externalize all config via env vars or mounted config files
- Use .dockerignore to exclude unnecessary files from build context
- Push to container registry, scan for vulnerabilities (Trivy, Snyk), then deploy

Q18: Your application is going live. How can Spring Boot Actuator be customized to provide detailed health metrics?

- Implement HealthIndicator for each critical dependency: database, Redis, Kafka, external APIs
- Implement ReactiveHealthIndicator for reactive (WebFlux) applications
- Include detailed health info in response: management.endpoint.health.show-details=always
- Register custom Micrometer metrics using MeterRegistry

```
meterRegistry.gauge("active.orders",
orderService,
OrderService::getActiveCount);
```

- Custom health groups: liveness (app can serve traffic) vs readiness (dependencies ready)

```
management.endpoint.health.group.liven
```

```
ess.include=ping
management.endpoint.health.group.readi
ness.include=db,redis
```

- Expose Prometheus metrics for Grafana dashboards

Q19 ⭐ [FREQUENTLY ASKED]: You need to integrate Kafka to handle real-time notifications in a social media application built with Spring Boot. How would you set up this integration?

- Add spring-kafka dependency (included in spring-boot-starter)
- Configure bootstrap servers and serializers:

```
spring.kafka.bootstrap-
servers=localhost:9092
spring.kafka.producer.key-
serializer=StringSerializer
spring.kafka.producer.value-
serializer=JsonSerializer
```

- Producer: Inject KafkaTemplate<String, NotificationEvent> and call kafkaTemplate.send(topic, event)
- Consumer: @KafkaListener(topics = "notifications", groupId = "notif-service")
- Create topics: Use @Bean NewTopic or admin client at startup
- Handle failures: Configure retry template, DLT (Dead Letter Topic) with @RetryableTopic
- Test with EmbeddedKafka in tests: @EmbeddedKafka(topics = "notifications")

Q20: How would you ensure that your custom starter doesn't interfere with Spring Boot's own auto-configuration?

- Use @ConditionalOnMissingBean — only configure if user hasn't defined their own bean
- Use @ConditionalOnClass — only apply auto-config if the library is on classpath
- Use @ConditionalOnProperty — allow users to disable your auto-config via property
- Use @AutoConfigureAfter / @AutoConfigureBefore to control ordering relative to Spring Boot's own configs

- Test with @SpringBootTest to verify your config doesn't break existing Spring Boot behavior
- Provide spring.factories exclusion path so users can exclude your starter if needed
- Use unique property prefixes to avoid collisions with Spring Boot's own properties

Q21: You're tasked with creating a custom starter for handling cloud-based message queuing across multiple microservices. What key components will you include?

- @ConfigurationProperties class: Queue URL, credentials, retry config, timeout settings
- Auto-configured MessageQueueClient bean with connection pooling
- MessagePublisher and MessageConsumer abstractions for the business layer
- Health indicator: Checks connectivity to the queue service
- Micrometer metrics: Messages sent, received, failed, processing time
- Retry and dead-letter queue handling with configurable retry policy
- Auto-configure the listener container factory with proper error handlers
- Test utilities: Embedded/mock queue for testing without real cloud dependency

Q22 ⭐ [FREQUENTLY ASKED]: Suppose you need to ensure zero downtime deployments for a Spring Boot e-commerce application. What approach would you take?

- Blue-Green Deployment: Run two identical environments; switch traffic via load balancer
- Rolling Deployment: Replace instances one at a time; K8s maxSurge/maxUnavailable control pace
- Canary Deployment: Route small % to new version; gradually increase on success
- Database: Backward-compatible schema changes (expand-contract);

never break old version

- API versioning: Old API endpoints remain during transition period
- Actuator readiness probe: K8s waits for /actuator/health/readiness=UP before routing traffic
- Graceful shutdown: spring.lifecycle.timeout-per-shutdown-phase=30s to drain in-flight requests

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=30s
```

Q23: What are the basic commands to create a Docker image and where are you storing your Docker images?

- Build image: docker build -t myapp:1.0.0 .
- Tag for registry: docker tag myapp:1.0.0 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0.0
- Push to registry: docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0.0
- Using Spring Boot plugin: mvn spring-boot:build-image -Dspring-boot.build-image.imageName=myapp:1.0.0
- Storage: AWS ECR (for AWS deployments), Azure ACR, Docker Hub (public), Artifactory/Nexus (private on-prem)
- Immutable tags: Use git SHA or version number, never overwrite the 'latest' tag in production

Q24 ⭐ [FREQUENTLY ASKED]: What is the use of Sleuth in Spring Boot microservice application? As Spring Boot 3.x no longer supports Sleuth, what is the alternative?

Spring Cloud Sleuth (deprecated):

- Automatically added traceId and spanId to MDC and log messages in each microservice
- Propagated trace context across HTTP calls, messaging, and async boundaries
- Integrated with Zipkin for distributed trace visualization

Spring Boot 3.x Alternative — Micrometer Tracing:

- Micrometer Tracing replaces Sleuth with micrometer-tracing-bridge-brave or -otel
- spring-boot-starter-actuator + micrometer-tracing provides auto-configuration
- Integrates with OpenTelemetry (OTel) Collector and Zipkin

```
management.tracing.sampling.probabilit
y=1.0
```

Q25 ⭐ [FREQUENTLY ASKED]: What is Micrometer Tracing in Spring Boot 3?

- Micrometer Tracing is the successor to Spring Cloud Sleuth in Spring Boot 3.x
- Provides a vendor-neutral tracing API (similar to how SLF4J abstracts logging)
- Bridges to brave (Zipkin) or OpenTelemetry implementations
- Automatically instruments: @RestController, RestTemplate, WebClient, @Async, @Scheduled, Kafka
- TraceId and SpanId are automatically added to logs and propagated via HTTP headers

```
// Programmatic span creation
Span span =
tracer.nextSpan().name("processOrder")
.start();
try (Tracer.SpanInScope ws =
tracer.withSpan(span)) {
  // traced code
} finally { span.end(); }
```

- Export traces to Zipkin, Jaeger, or any OpenTelemetry-compatible backend

Q26 ⭐ [FREQUENTLY ASKED]: How will you enable and disable auto-configuration in Spring Boot?

Enable (default — auto-enabled):

- @SpringBootApplication includes @EnableAutoConfiguration by default
- Add library to classpath → its auto-configuration is picked up automatically

Disable specific auto-configurations:

- Exclude via annotation: @SpringBootApplication(exclude = {DataSourceAutoConfiguration.class})
- Exclude via properties: spring.autoconfigure.exclude=...DataSo

urceAutoConfiguration

- Conditional annotations: @ConditionalOnProperty(name="feature.enabled", havingValue="true")
- If property is false/absent, the entire auto-configuration class is skipped

```
spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.security.servlet.SecurityAutoConfiguration
```

Q27 ⭐ [FREQUENTLY ASKED]: What is traceId and spanId in Spring Boot microservice application and what is the use of these IDs?

These are distributed tracing identifiers:

- TraceId: Unique ID for an entire end-to-end request across multiple microservices — same throughout the whole call chain
- SpanId: Unique ID for a single unit of work (one service call or operation) within a trace

Usage:

- Correlate logs across services — search traceId in Kibana to see all logs for one request
- Identify bottlenecks: Which service/span is slow in the distributed call chain
- Propagated via HTTP headers: traceparent (W3C format) or X-B3-TraceId (Zipkin format)
- Parent-child relationships: Each service creates a child span under the parent span

```
// Appears in logs automatically:
[traceId=abc123, spanId=def456] INFO - Processing order
```

Q28 ⭐ [FREQUENTLY ASKED]: What is WebFlux and Mono in Spring Boot?

Spring WebFlux:

- Reactive web framework introduced in Spring 5 — alternative to Spring MVC
- Non-blocking, event-loop based (Netty) — handles high concurrency with fewer threads
- Based on Project Reactor — uses reactive streams specification

Mono<T>:

- Represents a single async value (0 or 1 item) — like CompletableFuture

```
Mono<User> findById(Long id); // 0 or 1 user
```

Flux<T>:

- Represents a stream of async values (0 to N items) — like a reactive List

```
Flux<User> findAll(); // 0 to many users
```

- Neither executes until subscribed — they are lazy (cold publishers by default)

Q29 ⭐ [FREQUENTLY ASKED]: How will you create a custom annotation?

Step 1: Declare the annotation:

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface AuditLog {
  String action() default "";
}
```

Step 2: Create an AOP Aspect to process it:

```
@Aspect @Component
public class AuditLogAspect {
  @Around("@annotation(auditLog)")
  public Object
log(ProceedingJoinPoint pjp, AuditLog
auditLog) throws Throwable {
    log.info("Action: " +
auditLog.action());
    return pjp.proceed();
  }
}
```

Step 3: Use it:

```
@AuditLog(action = "CREATE_ORDER")
public Order createOrder(OrderRequest
request) { ... }
```

- Meta-annotations: Combine multiple Spring annotations into one custom annotation

Q30 ⭐ [FREQUENTLY ASKED]: How will you make two ambiguous URLs work in Spring Boot without changing the HTTP method type and without any change in URLs?

Scenario: Two endpoints with same URL and HTTP method — differentiated by request content.

- Use consumes attribute to differentiate by Content-Type:

```
@PostMapping(value = "/data", consumes
= "application/json")
```

```
public Response
handleJson(@RequestBody JsonRequest
req) { ... }

@PostMapping(value = "/data", consumes
= "application/xml")
public Response handleXml(@RequestBody
XmlRequest req) { ... }
```

- Use produces attribute to differentiate by Accept header:

```
@GetMapping(value = "/report",
produces = "application/pdf")
public byte[] getPdf() { ... }

@GetMapping(value = "/report",
produces = "application/json")
public ReportData getJson() { ... }
```

- Use headers attribute: @GetMapping(value="/data", headers="X-Api-Version=1")
- Use params attribute: @GetMapping(value="/search", params="type=advanced")

Quick Reference Cheat Sheet

Essential Spring Boot annotations and configuration properties for rapid revision before interviews.

Annotation / Property	Purpose / Usage
`@SpringBootApplication`	Entry point: @Configuration + @EnableAutoConfiguration + @ComponentScan
`@RestController`	Returns JSON responses; @Controller + @ResponseBody combined
`@Service / @Repository / @Component`	Stereotype annotations for business, data, and generic layers
`@Autowired`	Dependency injection (prefer constructor injection)
`@Qualifier / @Primary`	Resolve ambiguous bean injection
`@Value`	Inject property values: @Value("${app.name}")
`@ConfigurationProperties`	Bind a block of properties to a POJO; use with @Validated
`@Transactional`	Declarative transaction management; customize propagation and isolation
`@Async + @EnableAsync`	Run method in background thread
`@Scheduled + @EnableScheduling`	Cron or fixed-rate recurring tasks
`@Cacheable / @CacheEvict / @CachePut`	Method-level caching with Spring Cache abstraction
`@Profile`	Activate beans conditionally per environment profile
`@Valid / @Validated`	Trigger Bean Validation on controller parameters
`@MockBean / @Mock`	@MockBean for Spring context; @Mock for pure unit tests
`@WebMvcTest`	Test controller layer only (no service/repo)
`@DataJpaTest`	Test JPA repositories with embedded DB
`@SpringBootTest`	Load full application context for integration tests
`@PreAuthorize`	Method-level security with SpEL expressions
`@KafkaListener`	Consume Kafka messages from a topic
`@RequestBody / @ResponseBody`	Deserialize request JSON / serialize response to JSON
`Mono<T> / Flux<T>`	Reactive types: single value vs. stream of values (WebFlux)
`spring.profiles.active`	Activate environment profiles
`spring.autoconfigure.exclude`	Disable specific auto-configurations
`management.endpoints.web.exposure.include`	Expose Actuator endpoints
`spring.flyway.enabled`	Enable Flyway database migration
`server.shutdown=graceful`	Allow in-flight requests to complete on shutdown
`spring.jpa.hibernate.ddl-auto`	Schema generation: none/validate/update/create-drop
`spring.datasource.hikari.*`	HikariCP connection pool configuration
`management.tracing.sampling.probability`	Distributed tracing sample rate (1.0 = 100%)
`spring.session.store-type=redis`	Store HTTP sessions in Redis for clustering

Frequently Asked Questions Summary

#	Frequently Asked Question
1	How does Spring Boot auto-configuration work?
2	What testing strategies do you recommend?
3	How do you achieve multiple DB connections?
4	What mechanisms does Spring Boot provide for transaction management?

5	How do you handle form validation?
6	What are the ways to deploy a Spring Boot application?
7	How do you achieve logging in Spring Boot?
8	What is the difference between @Mock and @MockBean?
9	What is the @Async annotation?
10	What are alternatives of @Autowired?
11	Can you list Actuator endpoints?
12	What is the order of precedence in Spring Boot configuration?
13	How do you support reactive programming in Spring Boot?
14	What is the Spring Boot @Profile annotation used for?
15	How does method security work?
16	What is the CAP theorem?
17	How do you ensure zero downtime deployments?
18	What is Micrometer Tracing in Spring Boot 3?
19	What is traceId and spanId?
20	How will you create a custom annotation?

Best of luck with your Spring Boot interviews! □ □